

Teachers, Lessons and Students

Mohamed Kamal — your brief. Everything the client wants is on these six pages. How it is built is not on them, and that is on purpose.

The client

A platform where teachers teach and students learn, and **three different kinds of person** sign in to it.

An **administrator** lets teachers in. A **teacher** registers, waits to be let in — **or turned away** — then keeps their own lessons. Each carries a recording, and may carry a handout, a quiz and the answers to it. The teacher decides **when the lesson opens, when its quiz opens, and when its answers are released**: three separate moments, all theirs. They hold a **joining code**.

A **student registers on their own and belongs to nobody** — an account and a profile, no course. **Entering a teacher's code is a separate thing they do**, and it is what joins them — **and they can do it more than once**, so a student may be on several courses at a time. Inside each they see the lessons open to them now, are **told what is new** since they last looked *at that course*, and can ask a **helper** where to find things.

Four things this app is not. It does not host, upload or convert files — the recording and the handout are both *links*. It has no quiz engine: no question banks, no options, no auto-marking. There is no password reset, and no notification leaves the app — both need email. And the helper is not an AI: it is a list of phrases you write, matched to screens.

The moment you demo. Approve a teacher, add a lesson whose quiz opens tomorrow, then: **register a student who belongs to nobody — your API refuses them a course** · enter the teacher's code and it appears · **the recording plays and the quiz does not** · move the date to now and it does · **enter a second teacher's code and they are on two courses**, each with its own notices. Then ask for a course they never joined — **refused**.

What you must decide

None of it is written down for you anywhere: your **tables** and what one row of each is · the **relationships**, and what happens on delete · which table you prove first · an **endpoint table** with every failure code *and which of the three people may call it* · your routes and screens · every validation rule and the message a human reads.

What must work

Twenty-three outcomes, each carrying the way it is allowed to **fail**. Both halves are graded. **Your cut list matters more here than the list does.**

- 1 **A teacher registers themselves** — name, email, password — and lands on a screen saying the account is waiting.
 An email already in use **by anybody — teacher, student, or the administrator** — 400. A weak or mismatched password is caught before the server.
- 2 **Everyone signs in through one screen** — one email box, one password box, and **your server works out which of the three is asking**. The session survives a refresh; the token rides every call.
 A wrong password says so without saying which half. Signed out, the API answers 401. The three land in three different places.
- 3 The administrator **sees who is waiting** and **approves or refuses** each — **a decision that cannot be taken twice**. An unapproved teacher is refused everything but their own standing.
 Nobody waiting says so, and those screens are unreachable by anyone else. The refusal comes from the **server**. A turned-away teacher is *told*. Deciding twice is refused.
- 4 **A student registers on their own**, belonging to no teacher at all, and lands on their profile.
 An email already in use **by anybody** — 400. Nothing on the next screen pretends there is a course.
- 5 **A student joins a course by entering a teacher's code** — a separate act from registering, and **they may do it again for another teacher**.
 An unknown code, or one whose teacher is not approved — refused. The same code twice says "you are already on this course".
- 6 **A student's own profile**: their details, **every course they are on** and when they joined, and the parts they may change.
 What they may not change is not on the form — *and* the server refuses it anyway. Somebody on no courses is told so, and pointed at the joining screen.
- 7 **The home page, before anyone signs in** — what this is, how many teachers are on it, how to join.
 It reads correctly against an empty database, and needs no token.
- 8 The teacher adds, lists, edits and deletes a **lesson**: title, where it sits in the order, the recording link, **optionally a handout, a quiz and its answers**, the length, the quiz maximum and the pass mark.
 An empty title, a maximum of zero or less, a pass mark above the maximum, or a place another lesson holds — 400. Delete asks first, and is refused if marks exist.

What must work

- 9** The teacher **sets the three moments on a lesson** — when it opens, when its quiz opens, when the answers are released — and may leave any unset.

A quiz opening before its lesson does, or answers before the quiz — refused, saying *which pair* is wrong. **And a moment set on something that is not there is refused too:** an opening time for a quiz this lesson does not have is a promise it cannot keep.
- 10** The teacher's **lesson list, in its real order**, each showing which of its three moments have passed.

A teacher with no lessons yet says so, rather than a blank panel.
- 11** The teacher's **student list** — everyone who joined with their code — and the code somewhere they can copy it.

No students yet says so, and says how somebody joins.
- 12** **Record a mark:** which student, which lesson, what they scored.

The reason lands next to the field that caused it, submit stays disabled while invalid, and a student who is not theirs cannot be chosen.
- 13** The teacher **opens one student** and sees every lesson they have been marked on, in order, passed or failed against that lesson's own pass mark — and can correct a mark there.

A student id that does not exist — **404**, and a "not found" screen rather than a crash.
- 14** **The class summary:** every student, and how far through the course they have got, worked out from the marks themselves.

A teacher with no marks yet reads zero. It does not vanish and it does not spin forever.
- 15** **The courses a student is on, then one of them:** that teacher's lessons open to them *now*, in order, the recording **playing inside the page**, the handout where there is one, and their own mark on each.

A lesson with no link, or one that will not embed, shows a message and a plain link — never a dead grey box. A course whose teacher has opened nothing yet says so.
- 16** The student is handed **the quiz once its moment has passed, and the answers once theirs has** — neither one second before.

A quiz not open yet says when it opens, or says nothing at all. It never shows a dead control hinting at what is coming, and a lesson with no quiz says nothing about quizzes.

What must work

- 17 **The student is told what is new** since they last looked — per course and totalled — which lessons, quizzes and answers opened, *named* and attributed to their teacher.
 A first visit says welcome, not "12 new". Opening it twice says nothing is new. **Opening one course does not silence another.**
- 18 **The helper, for students.** Ask in plain words — "where are my results" — and get a sentence back and a link to the right screen.
 A question it does not know *says so*, and lists what it does know. Somebody on no courses is offered the joining screen, never a course.
- 19 **Your server enforces every one of those moments.** Anything whose moment has not come is **not in the response at all** — not sent and hidden, not sent and disabled. The browser is not what withholds it.
 Read the raw answer in Postman. An unopened lesson is **refused**; an unopened quiz or answer sheet is simply **not there**. If it is in the response, this is not built — however the screen looks.
- 20 **Your server keeps everyone inside their own data.** A teacher reaches only their own lessons, students and marks. A student reads only their own marks, and only the lessons of a teacher **they have actually joined** — the URL may say *which course*, but only the token says *who is asking*.
 A course they never joined, another teacher's lesson, another student's marks — **refused**, not an empty list. Same for a mark posted for somebody who is not *your* student.
- 21 **Your server refuses a student for a course they are not on** — whether they joined nothing at all, or simply not that teacher.
Refused, saying what to do. *Not* an empty list — that tells somebody never entitled to ask that the course is empty — and not a crash on a missing row.
- 22 The server refuses a **second mark for the same student on the same lesson**, and a score **outside what that lesson is marked out of** — the bound coming from the lesson, not your code.
400 on both, server-side. A correction edits the mark already there. Passed or failed is never sent up by the browser.
- 23 **The awkward cases**, everywhere data is loaded.
 Every list has **loading**, **error** and **empty**. **Stop the API mid-demo** and the screen says so — on every screen, not the one you rehearsed.

Who can do what

WHO	WHAT THEY CAN DO
Administrator	Lets teachers in, or refuses them. Owns nothing else — no lessons, no students, no marks.
Teacher , approved	Everything inside their own teaching, and nothing belonging to another teacher.
Teacher , waiting	Signs in, sees one sentence. Every other call answers 403 .
Teacher , turned away	Signs in, and is told they were refused <i>and when</i> . Every other call answers 403 .
Student , on one or more courses	<i>Reads</i> their teachers' open lessons, their own marks, what is new. <i>Writes exactly two things</i> : the course they join, and their own profile. Never a lesson, never a mark.
Student , on none yet	Their profile, the joining screen, the helper. Every course call answers 403 .

Three kinds of person and no fourth. Two register themselves; the third never did. Signed out, only the home page and the two registration forms answer — everything else is a **401**. **One email belongs to one person** across all three, the administrator's own address included, and no single table can promise that.

The stack is yours to pick

- **Recommended:** Angular, a .NET API, EF Core + SQLite. `AddCors` before `UseCors`.
- **Allowed:** any other stack or package — say so on day 16. Off the taught one, help thins fast.
- **Unchanged:** same must-haves, same demo, same JUDGING gate, whatever you pick.
- **Never:** uploads, payments, email or SMS, Google or Facebook login, websockets, background jobs.

Sign-in, and how far it goes

Your API holds the credentials, returns a JWT, `[Authorize]` guards what writes. Angular gets a login screen, a session service, an interceptor and guards. Two of your three people sign in on rows you already have — **no separate table of accounts**. Store passwords **hashed**, and **never send one back out**.

Out of scope, and say it out loud

No quiz engine, no password reset, no fourth kind of person. No subject or programme above a teacher — **a teacher is the course**. No student writing anything another person reads, no helper backed by a model or an API key, no attendance, chat or money. **And a hard limit on your table count**.

Your cut list is the plan

Features freeze **midday on day 18**. This brief is much larger than most, so before day 17 write down — *in order* — what you drop if you fall behind, and **expect to use it. Twelve outcomes finished beats twenty-three half-built**, and the marking scheme is written so that this is true. Mine runs to eleven lines.

AI writes, you own it

- Commit **before** every AI experiment. Git is the undo button.
- Read every diff. **Never merge a line you cannot explain** — one unexplained line in the core flow caps code ownership at 10 out of 25.
- Point, don't dump. Name the file. Never paste the repository.
- Tests over items 3, 19, 20 and 21 — how you check code you did not type.

The four days, and what you will be asked

- 16 Tables drawn by hand, endpoint table filled in with every failure code and who may call it, cut list ordered — all in `plan.md`, approved before any code exists.
- 17 **All three sign-ins work by the end of the day**, and one table travels the whole way: screen, API, database, still there after a refresh.
- 18 Cutting in the morning, freeze at midday, one timed practice run, and a stranger can run it from a clean clone.
- 19 The demo moment works live, and you can explain any three lines a judge points at.

Items 19, 20 and 21 decide who may see something — prove them in Postman, not a browser. Also: one screen takes an id in its URL and one does not, so why is that safe? · where is "open to a student" stored, and what happens when the teacher edits that date?

About this document. It is a picture, not text — anything reading it for you finds a notice where the requirements should be. Friction, not a wall: the wall is day 16, where you explain a design nothing in here gave you. Read it, think, write your own plan in your own words.